

Content Guide



Appendices

The following appendices contain information relative to a variety of Jupiter subjects. Use the table below to identify the section containing the information you are looking for.

Appendix	Description
A: About this Guide	Where to get additional help, and how to use this guide.
B: Localization Guidelines	Guidelines for localizing your game.
C: Console Variables	How to use the Console window to set variables used in your game.
D: API Issues	Describes specific API issues that may effect your game.
E: WorldModel Reference	Lists issues relating to the various WorldModels supplied with Jupiter.
F: FX Reference	Lists issues relating to the various FX supplied with Jupiter.
G: Glossary	Lists acronyms, terms, and file extensions related to Jupiter.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Appendix A: About This Guide

This Development Guide provides documentation to aid programmers, artists, and level designers to use Jupiter to create great games.

Other Documentation and Resources

LithTech maintains a web site for developer issues at <http://devnet.touchdownentertainment.com>. This website provides news, downloads, a release schedule, support, and a mailing list archive. Email support@touchdownentertainment.com for a user name and password to the web site.

Document Conventions

This Guide frequently references files installed from the LithTech Jupiter System CD to your hard drive. Whenever a file path is given in this guide, the root directory is your Jupiter installation directory. The default installation root directory is:

c:\lithtech\LT_Jupiter_Bin\ for binary licensees

c:\lithtech\LT_Jupiter_Src\ for source licensees.

The following typographic conventions are used in this book:

monospace—Monospace font identifies a syntax block of source code, examples, user input, file paths, or program output.

bold—Bold text identifies an interface, function, data type, macro, or other keyword from the SDK or a programming language.

italic—Italicized text signifies placeholders, such as parameters, in a syntax block. In your application implementation, these placeholders must be replaced with appropriate identifiers or values.

[]—Brackets encapsulate optional syntax items in syntax blocks. Type only the syntax contained within the brackets, not the brackets themselves.

—In the table of contents this icon indicates a page that contains links to other information, but does not contain content itself.

—In the table of contents this icon indicates links to information that resides on the same page as the folder above it. Each entry following this icon jumps you to the information specified by the text without taking you to a new page.

Setting Your Font Styles

When reading this document, be sure to set your default fonts to styles that promote online readability. Studies show that readers of online material find it easier to read styles that use sans serif fonts (such as Arial or Helvetica).

Note: .CHM Help files use components of the IE browser to create the .CHM Help interface. Because of this, adjustments you make in IE, such as colors, fonts, languages, and accessibility, apply to all of the .CHM Help systems on your entire computer.

You can change the fonts displayed by .CHM files by clicking the Options button and selecting Internet Options (which opens the IE Internet Options dialog box). Then, on the General tab, click the Fonts button and select the font (typeface) you wish to view in the Help system interface.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Appendix B: Localization Guidelines

You can use Jupiter to write a game that is easily localized for distribution in foreign countries. Here are some guidelines that will make it easier for you to localize your game.

- Move every text string that is displayed in the game into the CRES.DLL.
- All bitmap fonts in the game need a way to be turned off so that system TrueType fonts can be used instead.
- All the font names and sizes should be selectable in CRES.DLL so the localization company can adjust

them.

- All single key responses need to be selectable in CRES.DLL (for example, in a Yes/No dialog, yes is *oui* in French so the user needs to press the O key).
- All string manipulation must be done with the tchar (or mbcs) routines. Information about this is available in Visual C/C++ help. This allows the program to use multibyte character sets.
 - All projects should have the `_MBCS` flag set in the compiler directives.
- There should be a way to disable gore completely from the game in CRES.DLL so that the option to set the gore level is not even available to the user. Some countries will not allow gore.
- Anyplace where you use word wrapping, you must provide a way to deal with languages (such as Japanese) that do not have spaces between words. For these languages some type of hard wrapping where the line can be wrapped in the middle of a word must be used. This should be an option in CRES.DLL. Note that the rules for hard wrapping are not completely simple because certain characters should never be allowed to start or be in the first two characters of a line.
- If you are not localizing huge quantities of graphics and sounds, you can probably put all of your localized resources and a localized version of your CRES.DLL in a separate language .REZ file. It should be loaded last so its resources take priority. Usually you can then make patches and a new version of your software and still use the same localized .REZ file.
 - The default keyboard setup for a localized keyboard will probably be different so it will need to be re-configured in the AUTOEXEC.CFG file for each localized version. Sometimes the `##` keyboard codes must be used for this to work properly on foreign keyboards.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Appendix C: Console Variables

Console variables set a wide variety of parameters in a Jupiter game. A large number of these are very useful for debugging purposes. Others are used for communications settings and other engine-related behavior.

Console variable values are checked by Jupiter at various times depending on the specific console variable. Some are checked only at game initialization, and therefore changing them during the game will not have any effect on that game session. It may, however, effect future game sessions as the value is saved to the autoexec.cfg file. Others are checked every frame.

This appendix provides a list of the most useful console variables. Valid values listed in angle brackets. Default values are listed in parentheses. Use 1 for TRUE and 0 for FALSE.

Autoexec.cfg

The autoexec.cfg file is the repository for console variable values used in your game. This file must be located in the same folder as the LithTech.exe file.

You can set console variable values in this file in three ways:

1. Manually pre-set the values in a text editor and save it prior to game start up.
2. Use the Console to reset values while the game is running.
3. Programmatically change the values in your game code.

Jupiter overwrites the new values over existing values in the autoexec.cfg file when the game shuts down. Console

variables that did not exist in the autoexec.cfg file prior to game startup will not be added to the file even if they are set in the console or programmatically.

To pre-set the values for any console command in the .CFG file, use the following syntax:

command value

The **command** argument is the console command and the **value** argument is the value to which it is set.

The Console

For debugging purposes, console variables can be manually set during a running game using the Console. The Console is a text-based entry window in which you can assign console variable values. To access the console, press the TILDE (~) key in a running Jupiter application. In the Console you can directly access the client and server portions of Jupiter to give commands and to use debugging options.

Programmatic Console Variable Access and Modification

Console variables can be programmatically accessed and modified using a variety of functions. **HCONSOLEVAR**, defined in the iltserver.h header file, is a handle to a console variable.

The following functions are defined in the iltclient.h header file:

```
HCONSOLEVAR ILTClient::GetConsoleVar(char *pName);

LTRESULT ILTClient::GetSConValueFloat(char *pName, float &val);

LTRESULT ILTClient::GetSConValueString(char *pName, char *valBuf, uint32 bufLen);
```

The following functions are defined in the iltcsbase.h header file:

```
float GetVarValueFloat(HCONSOLEVAR hVar);

char* ILTCSBase::GetVarValueString(HCONSOLEVAR hVar);
```

The following functions are defined in the iltserver.h header file:

```
void (*SetGameConVar)(char *pName, char *pVal);

HCONVAR (*GetGameConVar)(char *pName);

void (*RunGameConString)(char *pString);
```

Input

AddAction—Sets a game action name to an integer. A game action is a name that defines a specific input action for a game. For instance, FirePistol could be a game action bound to a mouse click event.

EnableDevice—Enables an input device for recognition by Jupiter.

RangeBind—Binds specific inputs to specific commands.

Miscellaneous

ListCommands—Lists in the console all available commands.

LockPVS(0)—When set, locks the PVS (Potentially Visible Set). Useful for finding what is in your PVS at any given point in a level.

OptimizeSurfaces(0)—Enable/disable the creation of the tiles for optimized surfaces.

Quit—Exit the Jupiter game.

Serv—Prefix a console variable or command to run it on the server.

Worlds—Use this console command to start a new world. Specify the path to the world in the .rez file (or directory) but do not specify an extension.

Object Rendering Options

DrawCanvases(1)—Enable/disable the drawing of canvases.

DrawLineSystems(1)—Enable/disable the drawing of line systems.

DrawParticles(1)—Enable/disable the drawing of particles.

DrawPolygrids(1)—Enable/disable the drawing of poly grids.

DrawSky(1)—Enable/disable the drawing of the sky.

DrawSprites(1)—Enable/disable the drawing of sprites.

DrawWorldModels(1)—Enable/disable the drawing of world models.

Performance

ShowFrameRate—Prints the frame rate to the console.

ShowMemStats—Prints information about memory usage to the console.

ShowPolyCounts—Prints information about polygon drawing to the console.

ShowTickCounts<0-9>—Prints information about where time is being spent in the engine each frame.

Rendering Options

32bitTextures(0)—Enables selection of 32-bit Texture Format.

BitDepth—<16, 24, 32> (16) Screen Pixel Bit Depth. You must **RestartRender** for the new **BitDepth** to take effect. **BitDepth** will fall back to a lower value if necessary.

RestartRender—This is a console command that will recreate the renderer and all data associated with it. This must be run after any changes to the other Rendering Options to make them take effect.

ScreenHeight(480)—Screen Height. You must **RestartRender** for the new **ScreenHeight** to take effect.

ScreenWidth(640)—Screen Width. You must **RestartRender** for the new **ScreenWidth** to take effect.

World Rendering Options

WireFrame(0)—When true, draws the world and objects in WireFrame mode.

ShowSplits(0)—Shows the polygon splits on polygons in the world as a different color when **DrawFlat** is on.

DrawCurrentLeaf(0)—Highlights the polygons that form the current leaf of the BSP. As the player moves around the world, the highlighting reveals how the world's hulls are organized.

DrawFlat(0)—Draws all of the polygons in the world as a flat color.

Saturate(0)—Use Saturate blend for World Geometry.

LightMapsOnly(0)—Draws only the LightMaps and not the color texture on the world.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Appendix D: API Issues

The following issues may cause some abnormal behavior and should be accounted for in your game code.

- [HMODELANIM](#)
- [LT_NO and LT_YES](#)

HMODELANIM

The data type **HMODELANIM** is not a handle, but an index. So, zero is a perfectly valid value. In cases where the engine returns an **HMODELANIM** an invalid animation is represented by **((HMODELANIM)-1)**. So, don't write code that looks like:

```
HMODELANIM hAnim;

...

if (hAnim)

{

    // This code will NOT be evaluated when hAnim is set
    // to the first animation specified in the model.

    ...

}
```

[Top](#)

LT_NO and LT_YES

Beware of engine functions that use the LTRESULT values **LT_NO** and **LT_YES**. **LT_NO** is NOT defined as zero, and **LT_YES** is NOT defined as 1. In fact **LT_YES = 86** and **LT_NO = 87**. Make sure you write code to check for the appropriate return value and do not write code that does the following:

```
ILTModel* pLTModel;

...

// INCORRECT:
```

```

        if (pLTModel->GetLooping(hObj, nTrackerID))

            {

                // This code will ALWAYS be evaluated

                ...

            }

        //CORRECT:

        if (LT_YES == pLTModel->GetLooping(hObj, nTrackerID))

            {

                // This code will only be evaluated when the model is looping...

                ...

            }

```

This may cause some bugs because the API looks like it is returning a Boolean value in many of these cases. Here are the API interfaces that use these return types:

- ILTModel::GetLooping
- ILTModel::GetPlaying
- ILTModel::IsCollisionObjectsEnabled
- ILTModel::IsUpdateCollisionObjectsEnabled
- ILTModel::IsTransformCacheEnabled
 - CLTCursorInst::IsValid
- CLTCursor::IsCursorModeAvailable
 - CLTCursor::IsValidCursor
- ILTPhysics::IsWorldObject
- ILTPhysics::IsWorldObject

ILTPhysics::IsWorldObject checks to see if the passed in object is the **MAIN** world object, not just any world object. A better function to use to determine if the passed in object is the main world object is the function **IsMainWorld**, defined in CommonUtilities.h.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Appendix E: WorldModel Reference

This appendix provides information on the various WorldModel objects provided with Jupiter.

- [WorldModel](#)
- [RotatingWorldModel](#)
- [SlidingWorldModel](#)
- [SpinningWorldModel](#)

- [RotatingDoor](#)
- [SlidingDoor](#)
- [RotatingSwitch](#)
- [SlidingSwitch](#)

WorldModel

This is the basic non-active WorldModel object. It does not move or rotate, does not play any sounds and cannot execute any commands. It can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected.

Use this object any place where you need a stationary WorldModel. An example would be a desk that you want to damage or to be able to handle attachments. This object is also very well suited for keyframing. If you want a keyframed WorldModel object you should use this object.

Creation	This object is created normally. Simply bind this object to a brush or groups of brushes and then edit the properties as you see fit.
Damage	This object can handle damage. Edit the DamageProperties subset to define behavior.
Attachments	This object can handle attachments. Enter all the objects you want attached to the WorldModel, separated by a semicolon, in the Attachments property.
States	None. This object does not move or rotate so there is no need for states.
Messages	WorldModels can receive the following messages: • ATTACH Attaches the object specified in the message • DETACH Detach the object attached with the ATTACH message.
Options	None. The player and AI cannot activate this object. It has no movement or rotation properties.
Commands	None. This object cannot execute any commands.
Sounds	None. This object cannot play any sounds.
Animated Lightmaps	This object does not support animated light maps.

[Top](#)

RotatingWorldModel

This WorldModel can rotate around a point a specified number of degrees around each axis. The player can interact with this object or it can be controlled only through other objects. Sounds can be played and commands can be executed when certain states are reached. It can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected. This WorldModel can also have an animated light map associated with it.

Use this object any place you want a WorldModel that should swing like it is hinged to a wall. Kitchen cabinets or window shutters are a good examples of appropriate uses for a RotatingWorldModel.

Creation	This object is created normally. Simply bind this object to a brush or group of brushes. To set its rotation properties just edit the vector labeled RotationAngles and either position the bound object where you would like to rotate around or link to a Point object positioned where you want the object to rotate around. Enter the number of degrees, positive or negative, around each axis you would like this object to rotate. Using the kitchen cabinet as an example you would create your brush in the closed position, bind a RotatingWorldModel to the brush, move the bound object to either the left or right edge of the brush, and then edit the vector (0.0 140.0 0.0). This cabinet door will now open out 140 degrees.
Damage	This object can handle damage. Edit the DamageProperties subset to define behavior.
Attachments	This object can handle attachments. Enter all the objects you want attached to the RotatingWorldModel, separated by a semicolon, in the Attachments property.
States	• Off By default RotatingWorldModels start in the Off position, and are considered Off while in this position. • On When the RotatingWorldModel is fully rotated to the degree amounts specified in the RotationAngles property, it is considered On. • PowerOff While rotating from the On position towards the Off position the RotatingWorldModel is in the PowerOff state. • PowerOn While rotating towards the On position it is considered to be in the PowerOn state.
Messages	A RotatingWorldModel can receive the following messages: • ATTACH Attaches the object specified in the message • DETACH Detach the object attached with the ATTACH message. • LOCK Locks the object. Once locked the object cannot be activated. • OFF If not already in the Off or PowerOff states, puts the WorldModel in the PowerOff state. • ON If not already in the On or PowerOn states, puts the WorldModel in the PowerOn state. • TRIGGER Toggles the state. Basically the same as when a player presses use against this object. If the WorldModel is in the On or PowerOn state it will immediately switch to PowerOff. If it's in the Off or PowerOff state, then it will switch to PowerOn. • UNLOCK Unlocks the object so it can be activated by the player and other messages.
Options	A RotatingWorldModel has the following editable options: • ForceMove When TRUE the RotatingWorldModel will rotate through the player and other objects. • Locked Toggles whether this object starts locked or not. • PlayerActivate Toggles if the Player can interact with the RotatingWorldModel or not. • RemainOn

If this is TRUE the RotatingWorldModel will stay in the On position until directly told to turn off, either by a player or a message.

- **RotateAway**

If TRUE the RotatingWorldModel will swing away from the player.

- **StartOn**

When TRUE the RotatingWorldModel is in the On position at load time.

- **TriggerOff**

Toggles whether or not the player can directly turn a RotatingWorldModel Off.

- **Waveform**

Defines how the object Rotates.

Commands

A RotatingWorldModel can send these commands when in the corresponding state.

- **LockedComand**
- **OffCommand**
- **OnCommand**
- **PowerOffComand**
- **PowerOnCommand**

Sounds

A RotatingWorldModel can play these sounds when in the corresponding state.

- **LockedSound**
- **OffSound**
- **OnSound**
- **PowerOffSound**
- **PowerOnSound**

Animated Lightmaps

This object supports animated light maps.

[Top](#)

SlidingWorldModel

This WorldModel can slide, or move, a specified number of units in a specified direction. The player can interact with this object or it can be controlled only through other objects. Sounds can be played and commands can be executed when certain states are reached. It can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected. This WorldModel can also have an animated light map associated with it.

Use this object any place you want a WorldModel that should move in a specific direction. Desk drawers are a good example of what these can be used for but of course there are many uses.

Creation

This object is created normally. Simply bind this object to a brush or group of brushes.

To set its movement properties just edit the vector labeled **MoveDir** and set the distance you would like this SlidingWorldModel to move to in the property labeled **MoveDist**.

A **MoveDir** vector of (0.0 1.0 0.0) will move the SlidingWorldModel along its own local Y axis, typically straight up. This vector can be edited to point in any direction you want (0.5 0.5 0.0) will move the SlidingWorldModel diagonally along its X and Y axis. The **MoveDist** property is the distance the SlidingWorldModel will travel in DEdit units.

Using the desk drawer as an example you would create your brush in the closed position, bind a SlidingWorldModel to the brush and then edit the **MoveDir** vector (0.0 0.0 1.0). Now specify how far you want the object to move by making **MoveDist** 64.0. The desk drawer will now slide 64 units out in its local Z axis.

Damage	This object can handle damage. Edit the DamageProperties subset to define behavior.
Attachments	This object can handle attachments. Enter all the objects you want attached to the SlidingWorldModel, separated by a semicolon, in the Attachments property.
States	Off—By default SlidingWorldModels start in the Off position, and are considered Off while in this position. On—When the SlidingWorldModel is fully moved to the distance specified in the MoveDist property, it is considered On. PowerOff—While moving from the On position towards the Off position the SlidingWorldModel is in the PowerOff state. PowerOn—While moving towards the On position it is considered to be in the PowerOn state.
Messages	A SlidingWorldModel can receive the following messages. ATTACH—Attaches the object specified in the message DETACH—Detach the object attached with the ATTACH message. LOCK—Locks the object. Once locked the object cannot be activated. OFF—If not already in the Off or PowerOff states, puts the WorldModel in the PowerOff state. ON—If not already in the On or PowerOn states, puts the WorldModel in the PowerOn state. TRIGGER—Toggles the state. Basically the same as when a player presses use against this object. If the WorldModel is in the On or PowerOn state it will immediately switch to PowerOff. If it's in the Off or PowerOff state, then it will switch to PowerOn. UNLOCK—Unlocks the object so it can now be activated by the player and other messages.
Options	A SlidingWorldModel has the following editable options: ForceMove—When TRUE the SlidingWorldModel will move through the player and other objects. Locked—Toggles whether this object starts locked or not. PlayerActivate—Toggles if the Player can interact with the SlidingWorldModel or not. RemainOn—If this is TRUE the SlidingWorldModel will stay in the On position until directly told to turn off, either by a player or a message. StartOn—When TRUE the SlidingWorldModel is in the On position at load time TriggerOff—Toggles whether or not the player can directly turn a SlidingWorldModel Off. Waveform—Defines how the object moves.

Commands	A SlidingWorldModel can send the following commands when in the corresponding state: • LockedComand • OffCommand • OnCommand • PowerOffComand • PowerOnCommand
Sounds	A SlidingWorldModel can play the following sounds when in the corresponding state: • LockedSound • OffSound • OnSound • PowerOffSound • PowerOnSound
Animated Lightmaps	SlidingWorldModel objects support animated light maps.

[Top](#)

SpinningWorldModel

This WorldModel can spin around a point a in the specified amount of time to make one rotation around each axis. The player can interact with this object or it can be controlled only through other objects. Sounds can be played and commands can be executed when certain states are reached. Like all of the new objects this can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected. This WorldModel can also have an animated light map associated with it.

Use this object any place you want a WorldModel that should spin around a central point. Ceiling fans or a Rolodex are good examples of what these can be used for but of course there are many uses.

Creation	This object is created normally. Simply bind this object to a brush or group of brushes. To set its rotation properties just edit the vector labeled RotationAngles and either position the bound object where you would like to spin around or link to a Point object positioned where you want the object to spin around. Enter the amount of time, in seconds, to make one rotation around each axis, you would like this object to spin. Using the ceiling fan as an example you would create your brush in the closed position, bind a SpinningWorldModel to the brush, move the bound object to the center of the object (by default it is), and then edit the RotationAngles vector (0.0 4.0 0.0). This fan will now spin around its Y axis once every 4 seconds.
Damage	This object can handle damage. Edit the DamageProperties subset to define behavior.
Attachments	This object can handle attachments. Enter all the objects you want attached to the RotatingWorldModel, separated by a semicolon, in the Attachments property.
States	Off—By default RotatingWorldModels start in the Off position, and are considered Off when they are no longer spinning. On—When the SpinningWorldModel is spinning at the desired rate specified in the RotationAngles property, it is considered On. PowerOff—While spinning from the specified rate slowly towards a resting position

the SpinningWorldModel is in the PowerOff state.

PowerOn—While spinning around the point picking up speed towards the specified rate it is considered to be in the PowerOn state.

Messages

A SpinningWorldModel can receive the following messages:

ATTACH—Attaches the object specified in the message

DETACH—Detach the object attached with the ATTACH message.

LOCK—Locks the object. Once locked the object cannot be activated.

OFF—If not already in the Off or PowerOff states, puts the WorldModel in the PowerOff state.

ON—If not already in the On or PowerOn states, puts the WorldModel in the PowerOn state.

TRIGGER—Toggles the state. Basically the same as when a player presses use against this object. If the WorldModel is in the On or PowerOn state it will immediately switch to PowerOff. If it's in the Off or PowerOff state, then it will switch to PowerOn.

UNLOCK—Unlocks the object so it can now be activated by the player and other messages.

Options

A SpinningWorldModel has the following editable options:

ForceMove—When TRUE the SpinningWorldModel will rotate through the player and other objects.

Locked—Toggles weather this object starts locked or not.

PlayerActivate—Toggles if the Player can interact with the SpinningWorldModel or not.

RemainOn—If this is TRUE the SpinningWorldModel will keep on spinning until directly told to turn off, either by a player or a message.

StartOn—When TRUE the SpinningWorldModel will start spinning at load time

TriggerOff—Toggles weather or not the player can directly turn a SpinningWorldModel Off.

Waveform—Defines how the object Spins.

Commands

A SpinningWorldModel can send the following commands when in the corresponding state:

- **LockedComand**
- **OffCommand**
- **OnCommand**
- **PowerOffComand**
- **PowerOnCommand**

Sounds

A SpinningWorldModel can play these sounds when in the corresponding state.

- **LockedSound**

- **OffSound**
- **OnSound**
- **PowerOffSound**
- **PowerOnSound**

Animated Lightmaps

SpinningWorldModel objects support animated light maps.

[Top](#)

RotatingDoor

This Door can rotate around a point a specified number of degrees around each axis. The player can interact with this object or it can be controlled only through other objects. Sounds can be played and commands can be executed when certain states are reached. It can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected. This Door can also have an animated light map associated with it. Doors also have the ability to open another door they are linked to and they can open and close portals when they open and close.

Use RotatingDoor any place you want a Door that should swing like it is hinged to a wall. An office Door or a car Door are good examples of what these can be used for but of course there are many uses.

Creation

This object is created normally. Simply bind this object to a brush or group of brushes.

To set its rotation properties just edit the vector labeled **RotationAngles** and either position the bound object where you would like to rotate around or link to a Point object positioned where you want the object to rotate around. Enter the number of degrees, positive or negative, around each axis you would like this object to rotate.

Using the office door as an example you would create your brush in the closed position, bind a RotatingDoor to the brush, move the bound object to either the left or right edge of the brush, and then edit the vector (0.0 90.0 0.0). This office door will now open 90 degrees.

If you want this door to open another door whenever it is activated to open simply use the **DoorLink** property's **ObjectBrowser** to find the door name you would like. Typically Door1 will link to Door 2 and Door2 will link to Door1. This way doors right next to each other will open more naturally. If you want this door to control the opening and closing of a portal just type in the name of the portal brush you want in the **PortalName** property.

Damage

This object can handle damage. Edit the **DamageProperties** subset to define behavior.

Attachments

This object can handle attachments. Enter all the objects you want attached to the RotatingDoor, separated by a semicolon, in the **Attachments** property. A very good use for this would be a doorknob and a peephole.

States

Closed—By default a RotatingDoor starts in the Closed position, and are considered Closed whenever in this position.

Closing—While rotating from the Open position towards the Closed position the RotatingDoor is in the Closing state.

Open—When the RotatingDoor is fully rotated to the degree amounts specified in the RotationAngles property, it is considered Open.

Opening—While rotating towards the Open position it is considered to be in the Opening state.

Messages

A RotatingDoor can receive the following messages:

ATTACH—Attaches the object specified in the message

DETACH—Detach the object attached with the **ATTACH** message.

LOCK—Locks the object. Once locked the object cannot be activated.

OFF—If not already in the Closed or Closing states, puts the Door in the Closing state.

ON—If not already in the Open or Opening states, puts the Door in the Opening state.

TRIGGER—Toggles the state. Basically the same as when a player presses use against this object. If the Door is in the Open or Opening state it will immediately switch to Closing. If it's in the Closed or Closing state, then it will switch to Opening.

UNLOCK—Unlocks the object so it can now be activated by the player and other messages.

Options

A RotatingDoor has the following editable options:

AIActivate—Toggles if AI can interact with the door or not.

ForceMove—When TRUE the RotatingDoor will rotate through the player and other objects.

Locked—Toggles whether this object starts locked or not.

PlayerActivate—Toggles if the Player can interact with the RotatingDoor or not.

RemainOpen—If this is TRUE the RotatingDoor will stay in the Open position until directly told to close, either by a player or a message.

RotateAway—If TRUE the RotatingDoor will swing away from the player.

StartOpen—When TRUE the RotatingDoor is in the Open position at load time.

TriggerClose—Toggles whether or not the player can directly Close a RotatingDoor.

Waveform—Defines how the object Rotates.

Commands

A RotatingDoor can send the following commands when in the corresponding state:

- **LockedCommand**
- **ClosedCommand**
- **ClosingCommand**
- **OpenCommand**
- **OpeningCommand**

Sounds

A RotatingDoor can play the following sounds when in the corresponding state:

- **LockedSound**
- **ClosedSound**

- **ClosingSound**
- **OpenSound**
- **OpeningSound**

Animated Lightmaps

This object supports animated light maps.

[Top](#)

SlidingDoor

This Door can slide, or move, a specified number of units in a specified direction. The player can interact with this object or it can be controlled only through other objects. Sounds can be played and commands can be executed when certain states are reached. Like all of the new objects this can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected. This Door can also have an animated light map associated with it. Doors also have the ability to open another door they are linked to and they can open and close portals when they open and close.

Use SlidingDoor any place you want a Door that should move in a specific direction. Sliding glass doors or those sliding Asian doors are a very good example of what these can be used for, but of course there are many uses.

Creation

This object is created normally. Simply bind this object to a brush or group of brushes.

To set its movement properties, just edit the vector labeled **MoveDir** and set the distance you would like this SlidingDoor to move in the property labeled **MoveDist**. A **MoveDir** vector of (0.0 1.0 0.0) will move the SlidingDoor along its own local Y axis, typically straight up. This vector can be edited to point in any direction you want (0.5 0.5 0.0) will move the SlidingDoor diagonally along its X and Y axis. The **MoveDist** property is the distance the SlidingDoor will travel in DEdit units.

Using the sliding glass door as an example, you would create your brush in the closed position, bind a SlidingDoor to the brush and then edit the **MoveDir** vector (1.0 0.0 0.0). Now specify how far you want the object to move by making **MoveDist** 128.0. The sliding door will now slide 128 units out in its local X axis. If you want this door to open another door whenever it is activated, simply use the **DoorLink** property's **ObjectBrowser** to find the door name you would like. Typically Door1 will link to Door 2 and Door2 will link to Door1, this way doors right next to each other will open more naturally. If you want this door to control the opening and closing of a portal, just type in the name of the portal brush you want in the **PortalName** property.

Damage

This object can handle damage. Edit the **DamageProperties** subset to define behavior.

Attachments

This object can handle attachments. Enter all the objects you want attached to the SlidingDoor, separated by a semicolon, in the **Attachments** property. A very good use for this would be a doorknob and a peephole.

States

Closed—By default a SlidingDoor starts in the Closed position, and are considered Closed whenever in this position.

Closing—While moving from the Open position towards the Closed position the SlidingDoor is in the Closing state.

Open—When the SlidingDoor is fully moved to the distance specified in the MoveDist property, it is considered On.

Opening—While moving towards the Open position it is considered to be in the Opening state.

Messages

A SlidingDoor can receive the following messages:

ATTACH—Attaches the object specified in the message

DETACH—Detach the object attached with the **ATTACH** message.

LOCK—Locks the object. Once locked the object cannot be activated.

OFF—If not already in the Closed or Closing states, puts the Door in the Closing state.

ON—If not already in the Open or Opening states, puts the Door in the Opening state.

TRIGGER—Toggles the state. Basically the same as when a player presses use against this object. If the Door is in the Open or Opening state it will immediately switch to Closing. If it's in the Closed or Closing state, then it will switch to Opening.

UNLOCK—Unlocks the object so it can now be activated by the player and other messages.

Options

A SlidingDoor has the following editable options:

AIActivate—Toggles if AI can interact with the door or not.

ForceMove—When TRUE the SlidingDoor will rotate through the player and other objects.

Locked—Toggles whether this object starts locked or not.

PlayerActivate—Toggles if the Player can interact with the SlidingDoor or not.

RemainOpen—If this is TRUE the SlidingDoor will stay in the Open position until directly told to close, either by a player or a message.

StartOpen—When TRUE the SlidingDoor is in the Open position at load time.

TriggerClose—Toggles whether or not the player can directly Close a SlidingDoor.

Waveform—Defines how the object Moves.

Commands

A SlidingDoor can send the following commands when in the corresponding state.

- **LockedCommand**
- **ClosedCommand**
- **ClosingCommand**
- **OpenCommand**
- **OpeningCommand**

Sounds

A SlidingDoor can play the following sounds when in the corresponding state.

- **LockedSound**
- **ClosedSound**
- **ClosingSound**
- **OpeningSound**
- **OpenSound**

**Animated
Lightmaps**

This object supports animated light maps.

[Top](#)

RotatingSwitch

This Switch can rotate around a point a specified number of degrees around each axis. The player can interact with this object or it can be controlled only through other objects. Sounds can be played and commands can be executed when certain states are reached. It can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected. This Switch can also have an animated light map associated with it.

Use RotatingSwitch any place you want a Switch that should swing like it is hinged to the floor. Levers are a good example of what these can be used for but of course there are many uses.

Creation

This object is created normally. Simply bind this object to a brush or group of brushes.

To set its rotation properties just edit the vector labeled **RotationAngles** and either position the bound object where you would like to rotate around or link to a Point object positioned where you want the object to rotate around. Enter the number of degrees, positive or negative, around each axis you would like this object to rotate.

Using the lever as an example you would create your brush in the closed position, bind a RotatingSwitch to the brush, move the bound object to the bottom edge of the brush, and then edit the vector (0.0 0.0 -45.0). This lever will now rotate -45 degrees around its Z axis.

Damage

This object can handle damage. Edit the **DamageProperties** subset to define behavior.

Attachments

This object can handle attachments. Enter all the objects you want attached to the RotatingSwitch, separated by a semicolon, in the **Attachments** property. A good example of this would be a handle for the lever.

States

Off—By default RotatingSwitches start in the Off position, and are considered Off while in this position.

On—When the RotatingSwitch is fully rotated to the degree amounts specified in the RotationAngles property, it is considered On.

PowerOff—While rotating from the On position towards the Off position the RotatingSwitch is in the PowerOff state.

PowerOn—While rotating towards the On position it is considered to be in the PowerOn state.

Messages

A RotatingSwitch can receive the following messages:

ATTACH—Attaches the object specified in the message

DETACH—Detach the object attached with the ATTACH message.

LOCK—Locks the object. Once locked the object cannot be activated.

OFF—If not already in the Off or PowerOff states, puts the Switch in the PowerOff state.

ON—If not already in the On or PowerOn states, puts the Switch in the

PowerOn state.

TRIGGER—Toggles the state. Basically the same as when a player presses use against this object. If the Switch is in the On or PowerOn state it will immediately switch to PowerOff. If it's in the Off or PowerOff state, then it will switch to PowerOn.

UNLOCK—Unlocks the object so it can now be activated by the player and other messages.

Options

A RotatingSwitch has the following editable options:

ForceMove—When TRUE the RotatingSwitch will rotate through the player and other objects.

Locked—Toggles whether this object starts locked or not.

PlayerActivate—Toggles if the Player can interact with the RotatingSwitch or not.

RemainOn—If this is TRUE the RotatingSwitch will stay in the On position until directly told to turn off, either by a player or a message.

StartOn—When TRUE the RotatingSwitch is in the On position at load time

TriggerOff—Toggles whether or not the player can directly turn a RotatingSwitch Off.

RotateAway—If TRUE the RotatingSwitch will swing away from the player.

Waveform—Defines how the object Rotates.

Commands

A RotatingSwitch can send the following commands when in the corresponding state:

- **LockedCommand**
- **OffCommand**
- **OnCommand**
- **PowerOffCommand**
- **PowerOnCommand**

Sounds

A RotatingSwitch can play the following sounds when in the corresponding state.

- **LockedSound**
- **OffSound**
- **OnSound**
- **PowerOffSound**
- **PowerOnSound**

Animated Lightmaps

This object supports animated light maps.

[Top](#)

SlidingSwitch

This Switch can slide, or move, a specified number of units in a specified direction. The player can interact with

this object or it can be controlled only through other objects. Sounds can be played and commands can be executed when certain states are reached. It can handle Attachments, can have BlendModes, has Damage properties, and a surface type can be selected. This WorldModel can also have an animated light map associated with it.

Use SlidingSwitch any place you want a Switch that should move in a specific direction. Push buttons are a good example of what these can be used for but of course there are many uses.

Creation

This object is created normally. Simply bind this object to a brush or group of brushes.

To set its movement properties just edit the vector labeled **MoveDir** and set the distance you would like this SlidingSwitch to move to in the property labeled **MoveDist**. A **MoveDir** vector of (0.0 0.0 1.0) will move the SlidingSwitch along its own local Z axis, typically forward. This vector can be edited to point in any direction you want (0.5 0.5 0.0) will move the SlidingSwitch diagonally along its X and Y axis. The **MoveDist** property is the distance the SlidingSwitch will travel in DEdit units.

Using the push button as an example you would create your brush in the closed position, bind a SlidingSwitch to the brush and then edit the **MoveDir** vector (0.0 0.0 1.0). Now specify how far you want the object to move by making **MoveDist** 8.0. The push button will now slide 8 units out in its local Z axis.

Damage

This object can handle damage. Edit the **DamageProperties** subset to define behavior.

Attachments

This object can handle attachments. Enter all the objects you want attached to the SlidingSwitch, separated by a semicolon, in the **Attachments** property.

States

Off—By default SlidingSwitches start in the Off position, and are considered Off while in this position.

On—When the SlidingSwitch is fully moved to the distance specified in the **MoveDist** property, it is considered On.

PowerOff—While moving from the On position towards the Off position the SlidingSwitch is in the PowerOff state.

PowerOn—While moving towards the On position it is considered to be in the PowerOn state.

Messages

A SlidingSwitch can receive the following messages:

ATTACH—Attaches the object specified in the message

DETACH—Detach the object attached with the **ATTACH** message.

LOCK—Locks the object. Once locked the object cannot be activated.

OFF—If not already in the Off or PowerOff states, puts the Switch in the PowerOff state.

ON—If not already in the On or PowerOn states, puts the Switch in the PowerOn state.

TRIGGER—Toggles the state. Basically the same as when a player presses use against this object. If the Switch is in the On or PowerOn state it will immediately switch to PowerOff. If it's in the Off or PowerOff state, then it will switch to PowerOn.

UNLOCK—Unlocks the object so it can now be activated by the player and other messages.

Options

A SlidingSwitch has the following editable options:

ForceMove—When TRUE the SlidingSwitch will move through the player and other objects.

Locked—Toggles weather this object starts locked or not.

PlayerActivate—Toggles if the Player can interact with the SlidingSwitch or not.

RemainOn—If this is TRUE the SlidingSwitch will stay in the On position until directly told to turn off, either by a player or a message.

StartOn—When TRUE the SlidingSwitch is in the On position at load time

TriggerOff—Toggles weather or not the player can directly turn a SlidingSwitch Off.

Waveform—Defines how the object moves.

Commands

A SlidingSwitch can send the following commands when in the corresponding state:

- **LockedComand**
- **OffCommand**
- **OnCommand**
- **PowerOffComand**
- **PowerOnCommand**

Sounds

A SlidingSwitch can play the following sounds when in the corresponding state:

- **LockedSound**
- **OffSound**
- **OnSound**
- **PowerOffSound**
- **PowerOnSound**

Animated Lightmaps

This object supports animated light maps.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide

Appendix F: FX Reference

This section provides reference information for the following FX available in Jupiter.

- [CamJitter](#)
- [CreateFX](#)
- [DynaLight](#)

- [FlareSprite](#)
- [LTBModel](#)
 - [Null](#)
- [ParticleSystem](#)
- [PlayRandomSound](#)
 - [PlaySound](#)
 - [PolyTrail](#)
 - [Sprite](#)
- [SpriteSystem](#)
- [WonkyFX](#)

CamJitter

This FX will shake the camera around as though the player is in an earthquake.

This FX is good for simulating a shaky ground. When an explosion occurs it is a nice effect to shake the camera.

Key Options

- **Color Keys**

This FX does not allow you to edit its color keys.

- **Scale Keys**

This FX allows you to edit its Scale Keys. The Scale Keys for a CamJitter FX controls how intense the shaking is. This allows you to fade the shaking off rather than just abruptly stopping it.

- **Motion Keys**

This FX does not allow you to edit its Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **InnerRadius**

If the player is within this distance from the center of the FX they will feel the shaking at full force.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **OuterRadius**

If the player is between the OuterRadius and the InnerRadius they will feel an interpolated shake. If outside this distance the player will not feel the shake.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position:

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

CreateFX

This is a unique FX in that it doesn't render anything or really do anything but start another premade GroupFX. You can specify the name of a GroupFX and whether or not the new FX will loop so you can combine different GroupFX together quickly and easily. You have the option to use a blinding flare by editing the Blind properties. A blinding flare will disable the Z and be drawn over everything else if there is a clear path to the camera.

Whenever you want to start a preexisting GroupFX at a certain time within another GroupFX, you can use this FX to set it up. The new GroupFX will begin playing at the time specified by this FX's Key and will run independently of the GroupFX that contains it. This could mainly be used if you have an explosion that you want to play and then a looping cloud of smoke in the middle of the explosion to keep playing after the explosion finishes.

Key Options

- **Color Keys**

This FX does not allow you to edit its Color keys.

- **Scale Keys**

This FX does not allow you to edit its Scale keys.

- **Motion Keys**

This FX does not allow you to edit its Motion Keys.

Properties

- **FXName**

This is the name of the preexisting GroupFX that you wish to play.

- **Loop**

If set to Yes, the newly created GroupFX will loop and continue to play after the GroupFX that contains it is finished.

[Top](#)

DynaLight

This FX creates a Dynamic light with the color and scale specified. The color, scale and Intensity of the light can change over time.

The DynaLight FX can be used to show something is bright and emitting light. A good example of this is for a MuzzleFlash or for a fire or explosion.

Key Options

- **Color Keys**

This FX allows you to edit its color keys. Intensity of the light is controlled through the color keys by setting the Alpha. Fully Opaque is fully bright and Fully Translucent is No light. The Light's color and intensity will change over time by interpolating between the color keys.

- **Scale Keys**

This FX allows you to edit its Scale Keys. The Light's scale will change over time by interpolating between the scale keys. The scale is in DEdit units so for this FX the scale should be at least 100 to even show up. Because changing the scale of a dynamic light forces the engine to recalculate the lightmaps, it is better to

fade a DynaLight in and out using the intensity scale.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The DynaLight will follow the path of the Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **Flicker**

This will give the light a slight flicker feel to it by slightly changing the color and intensity.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position:

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

FlareSprite

The FlareSprite FX displays a sprite specified through a .SPR file and will scale and fade the sprite based on angles of the camera and sprite. The sprite can rotate, be flat against a wall, or always face the camera.

The most common use for a FlareSprite is as a “halo” of sorts for a light.

Key Options

- **Color Keys**

This FX allows you to edit its color keys. The Sprite’s color and alpha will change over time by interpolating between the color keys.

- **Scale Keys**

This FX doesn’t allow you to edit its Scale Keys.

- **Motion Keys**

This FX doesn’t allow you to edit its Motion Keys

Properties

- **AlongNormal**

The sprite will face in the direction specified by the Normal Property.

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **BlendMode**

Depending on the sprite used you may wish to set it to use Additive or Multiply blending.

- **BlindCameraAngle**

This is the angle from the Camera to the Sprite that is compared against for creating the blinding flare. If the angle is greater the BlindCameraAngle then the blinding flare is not created.

- **BlindMaxScale**

If the angle from the Sprite’s forward to the Camera is less than BlindSpriteAngle and if the angle from the Camera’s forward to the Sprite is less than BlindCameraAngle then the angle, specified by BlindObjectAngle, is used to calculate the scale of the sprite from MaxScale to BlindMaxScale.

- **BlindObjectAngle**

You can choose which object to use for the angle, sprite or camera, to calculate the scale.

- **BlindSpriteAngle**

This is the angle from the Sprite to the camera that is compared against for creating the blinding flare. If the angle is greater the BlindSpriteAngle then the blinding flare is not created.

- **CameraFacing**

The sprite will always be oriented so its forward is facing the camera.

- **DisableLight**

This controls whether or not dynamic lights affect the color of the sprite.

- **DisableZ**

When this is set to Yes the sprite will look as though it is being drawn over everything else.

- **Facing**

A dropdown list of presets that tell how the sprite will be oriented.

- **MaxAlpha**

When the angle is between 0 and MinAngle the alpha will go no larger than MaxAlpha.

- **MaxScale**

When the angle is between 0 and MinAngle the scale will go no larger than MaxScale.

- **MinAlpha**

When the angle is between 0 and the MinAngle the alpha will go no lower than MinAlpha.

- **MinAngle**

This is the minimum angle that the FX will use to calculate the scale, color and alpha of the Sprite. If the angle is less than this the sprite will interpolate its color, scale and alpha from 0, being the max, and MinAngle being the minimum. If the angle is greater than MinAngle, the FlareSprite is invisible.

- **MinScale**

When the angle is between 0 and the MinAngle the scale will go no lower than MinScale.

- **Normal**

This is the normal the sprite will be facing when set to face along the normal. By default this is set to be the forward vector of the sprite.

- **ObjectAngle**

This is a dropdown list to choose which object to use to calculate the angle. If Sprite is chosen the MinAngle is compared against the angle from the sprites forward to the camera. If Camera is chosen then the angle used is from the forward direction of the camera to the center of the sprite.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **Sprite**

This is the sprite (.SPR) file used for the Sprite FX. To select a .SPR file click the button and browse through the Resource Browser and select the desired file.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position.

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **ParentAlign**

If the FX has a parent then the sprite will have the same rotation as the parent object.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

LTBModel

This FX allows you to display a model that can fade in and out, scale, rotate and change the default animation length.

The LTBModel can be used as a chunk of something flying through the air (like a dead body) or you can set it up so many LTBModels are used as a gib FX.

Key Options

- **Color Keys**

This FX allows you to edit its Color keys. The Model's color and alpha will change over time by interpolating between the color keys. Alpha will only work on models with an additive RenderStyle associated with it.

- **Scale Keys**

This FX allows you to edit its Scale Keys. The Model's scale will change over time by interpolating between the scale keys.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The Model will follow the path of the Motion Keys.

Properties

- **AniLength**

If OverrideAniLength is set to Yes and this is non-zero then this length, in seconds, will be used to set the length of the animation. If this is 0.0 and OverrideAniLength is set to Yes, then the lifespan of the FX is used to set the length of the animation.

- **AttachName**

If any of the "Attach" options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **Facing**

A dropdown list of presets that tell how the sprite will be oriented.

- **CameraFacing**

The model will always be oriented so its forward is facing the camera.

- **AlongNormal**

The model will face in the direction specified by the Normal Property.

- **ParentAlign**

If the FX has a parent then the model will have the same rotation as the parent object.

- **Model**

This is the model (.LTB) file used for the LTBModel FX. To select an .LTB file click the button and browse through the Resource Browser and select the desired file.

- **Normal**

This is the normal the model will be facing when set to face along the normal. By default this is set to be the forward vector of the model.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **OverrideAniLength**

You can manually set the length of the animation or let the length of the FX determine the length by setting this to Yes.

- **RenderStyle 0-3**

The renderstyle properties are used to set renderstyles on the model. Each renderstyle should correspond to the renderstyle index of the model set in ModelEdit.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **Skin 0-4**

The skin properties are used to texture the model. Each skin should correspond to the texture index of the models textures set in model edit. To select a skin, click the button and select the .DTX or .SPR you wish to use.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position.

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

Null

This FX is used solely to give some random motion to other FX within a GroupFX. Nothing gets drawn for this FX; its only purpose is to keep track of a position for other FX to use. Other FX, a single FX or multiple FX, should motion link to this FX to give them a position to follow.

Other FX should motion link to this FX. It should not be used as a stand alone FX. You can motion link any FX to this Null FX to give them a random motion. PolyTrails should motion link to this FX to create sparks and trails.

You can link multiple FX within a GroupFX to this FX so it gives them a since of motion together.

Key Options

- **Color Keys**

This FX does not allow you to edit its Color keys.

- **Scale Keys**

This FX does not allow you to edit its Scale Keys.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The ParticleSystem will follow the path of the Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **Bounce**

If set to Yes, the FX will collide and bounce off geometry and solid objects in a realistic manner.

- **GravityAcceleration**

This controls the speed and direction the FX will float or sink. A negative value will make the FX sink to the ground and a positive value will float the FX up towards the sky. Gravity is always in relation to the world, no matter what the FX rotation is.

- **MaxVelocity**

This controls the maximum speed and direction the FX will have. The velocity is determined by getting a random value between the MinVelocity and MaxVelocity so each time the FX is played the motion will be different.

- **MinVelocity**

This controls the minimum speed and direction the FX will have in relation to its self and not the world.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

ParticleSystem

This is probably the most used FX. The particle system emits individual particles based on the properties that you fill out. This FX allows you to edit its Color Keys, Scale Keys and Motion Keys. The particles are always facing the camera and will never rotate. You can specify the particles to use a texture or a sprite. The Color Keys and Scale Keys for this FX are per particle, not per system. This means the particle will interpolate between its scale and color keys based on its own lifetime.

The ParticleSystems can be used almost anywhere for anything. They can be used as smoke from a steam pipe, fire, sparks and smoke from a gun and many other possibilities. You can set them up to be an impact FX and emit a puff of smoke when a bullet hits a wall, or as bubbles in the water.

Key Options

- **Color Keys**

This FX allows you to edit its color keys. The Color Keys for a Particle System are per particle. So every

particle will use the color keys based on their own lifespan.

- **Scale Keys**

This FX allows you to edit its Scale Keys. The Scale Keys for a Particle System are per particle. So every particle will use the scale keys based on their own lifespan.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The ParticleSystem will follow the path of the Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **BlendMode**

Depending on the texture or sprite used you may wish to set the system to use Additive or Multiply blending.

- **Bounce**

If set to Yes, each particle will collide with and bounce off geometry and solid objects in a realistic manner. This can dramatically decrease frames and slow the game. This should be used sparingly and only with systems that have few active particles.

- **EmissionInterval**

This controls the length of time, in seconds, between emissions of the system. Typically this is set low so the system is constantly emitting particles, however if you want particle to be emitted in bursts raise the value.

- **EmissionPlane**

This is the normal vector of the plane that the particles are emitted from. By default this is a plane parallel to the systems Up vector. If you are using a ParticleSystem as an impact FX this should be set to 0.0 0.0 1.0.

- **FlipRenderingOrder**

By default the newest particles are drawn first, over the older particles in the system. You can change the order so older particles are drawn over newer ones by setting this to Yes.

- **GravityAcceleration**

This controls the speed and direction the particles will float or sink. A negative value will make the particles sink to the ground and a positive value will float the particles up towards the sky. Gravity is always in relation to the world, no matter what the systems rotation is.

- **MaxParticleLifeSpan**

Each particle will last no longer than this amount of time, in seconds. The life span of each particle is a random number between the Min/MaxParticleLifeSpan. If you want every particle to last the same amount of time, make these exactly the same.

- **MaxParticleVelocity**

This controls the maximum speed and direction that each particle will have when they are emitted.

- **MaxRadius**

This is the maximum distance from the center of the system that the particles will be emitted.

- **MinParticleLifeSpan**

Each particle will last for at least this amount of time, in seconds.

- **MinParticleVelocity**

This controls the minimum speed and direction that each particle will have when they are emitted. The velocity is relative to the system so if you never want particles to be emitted in the downward direction of the system, make the Y component positive. For impact FX, the Z component should be positive so particles never get emitted into the surface they hit.

- **MinRadius**

This is the minimum distance from the center of the system that the particles will be emitted.

- **MoveParticlesWithSystem**

If a particle system is moving or rotating, the particles will move and rotate with the system. You can override this behavior by setting this property to No. For speed reasons, if a system is staying still (ie. Not moving or rotating), keep this value set to Yes.

- **Offset**

If the value of this vector property is anything besides zero, the FX will be placed at this position offset from its center.

- **ParticlesPerEmission**

Every time the system emits particles, based on the EmissionInterval, it will emit this many particles.

- **RotateAdd**

If the value of this property is anything besides zero, the FX will rotate this amount around the axis every update.

- **Sprite**

The ParticleSystem can use an .SPR file as its particles and each particle will loop through the textures in the sprite. To select a .SPR file, click the button and browse through the Resource Browser and select the desired file.

- **Texture**

If no Sprite is given, the ParticleSystem can use a single .DTX as its file for the particles to display. To select a .DTX file, click the button and select the file through the Resource Browser.

- **Type**

A dropdown list box of predefined emission types for the system:

- **Sphere**

The particles will emit in all directions around the center of the system. The emission distance will be no less than MinRadius from the center and no more than MaxRadius from the center.

- **Point**

The particles will emit from the same random point on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius from the center. If you wish to have the particles emit from the direct center of the system make Min/MaxRadius 0.0.

- **Circle**

The particles will emit in a circle on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius From the center. Playing with Min/MaxRadius can yield an emission pattern of a circle, doughnut, or ring.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position:

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

PlayRandomSound

Just like the PlaySound FX, except this FX will let you play a random sound from a selected set of sounds. You provide the base sound and then it randomly grabs sound with that base and a number appended at the end.

If there are several sounds available for an effect and you don't want to use just one for fear of being redundant, this is what you should use. This FX is good to use if a single GroupFX is used and played quite often to give it a more random and non set feel.

Key Options

- **Color Keys**

This FX doesn't allow you to edit its color keys.

- **Scale Keys**

This FX doesn't allow you to edit its scale keys.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The Sound will follow the path of the Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **InnerRadius**

If the player is within this distance from the center of the FX they will hear the sound at full volume.

- **Loop**

If this is set to Yes, then the sound will loop for the duration of the GroupFX.

- **NumRand**

This is how many sounds there are to use and the sound will be played by getting a random number between 0 and NumRand.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **OuterRadius**

If the player is between the OuterRadius and the InnerRadius they will hear an interpolated volume for the sound. If outside this distance the player will not hear the sound.

- **PitchShift**

This is a multiplier to the playback frequency of the sound, which will shift its pitch. If you had a sound recorded at 22,000 Hz and set its pitch shift to 1.1f, then the output sound would be 24,200 Hz and make the sound have a higher pitch.

- **PlayLocal**

If set to Yes, the sound will feel as though its playing in the players head.

- **Priority**

How important is this sound. If there are too many sounds playing at once, the lowest priority sounds are not played.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **Sound**

This is the file (.WAV) used as the base sound to play. To select a sound, click the button and select the .wav you wish to use.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position.

- Fixed

This is the most common option and will keep the FX at a fixed position.

- Follow

If the FX is motion linked to another FX UpdatePos must be set to follow.

- PlayerView

If you are using this FX as a MuzzleFlash, select PlayerView.

- NodeAttach

If this is selected the FX will attach to the specified node.

- SocketAttach

If this is selected the FX will attach to the specified socket.

- PV_NodeAttach

This will attach the FX to a node in a PlayerView model.

- PV_SocketAttach

This will attach the FX to a socket on a PlayerView model.

- **Volume**

This is the full volume of the sound heard within the InnerRadius.

[Top](#)

PlaySound

This FX allows you to play a sound at anytime during the GroupFX. You can control the radius, volume and whether or not the sound loops.

Any time you want a specific sound to play, like a fire crackling, or a steam vent, you can use this to play a sound.

Key Options

- **Color Keys**

This FX doesn't allow you to edit its color keys.

- **Scale Keys**

This FX doesn't allow you to edit its scale keys.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The Sound will follow the path of the Motion Keys.

Properties

- **AttachName**

If any of the "Attach" options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **InnerRadius**

If the player is within this distance from the center of the FX they will hear the sound at full volume.

- **Loop**

If this is set to Yes, then the sound will loop for the duration of the GroupFX.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **OuterRadius**

If the player is between the OuterRadius and the InnerRadius they will hear an interpolated volume for the sound. If outside this distance the player will not hear the sound.

- **PitchShift**

This is a multiplier to the playback frequency of the sound, which will shift its pitch. If you had a sound recorded at 22,000 Hz and set its pitch shift to 1.1f, then the output sound would be 24,200 Hz and make the sound have a higher pitch.

- **PlayLocal**

If set to Yes, the sound will feel as though its playing in the players head.

- **Priority**

How important is this sound. If there are too many sounds playing at once the lowest priority sounds are not played.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **Sound**

This is the file, .wav, used as the sound to play. To select a sound, click the button and select the .wav you wish to use.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

- **Volume**

This is the full volume of the sound heard within the InnerRadius.

[Top](#)

PolyTrail

This FX will draw a series of polygons following a path to create a motion trail behind the position the FX is following. This FX should be motion linked to another FX to give it more of a moving feel or attached to a node or socket of a model to give it a moving feel.

Motion link a PolyTrail FX to a NullFX and you can create sparks from a bullet hitting metal, or electrical sparks from the end of a downed power line. Attach this FX to a grenade or rocket to produce a motion trail or even a smoke trail behind it. Attach this FX to a socket on a model, like a sword, to show the path of motion.

Key Options

- **Color Keys**

This FX allows you to edit its color keys. The Color Keys for a PolyTrail are per trail section and not the trail as a whole. So the section closest to the head will be the color of the first key and the tail section will be the color of the last key.

- **Scale Keys**

This FX does not allow you to edit its Scale Keys.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The PolyTrail will follow the path of the Motion Keys. Allowing you to create the look of the trail.

Properties

- **AlphaTest**

A drop down list of various alpha tests to perform when determining whether or not to draw the trail.

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **BlendMode**

A drop down list of various blend mode operations that can be performed to get the desired effect.

- **ColorOp**

A drop down list of various color operations such as Additive and Multiply to perform when determining how to draw the trail.

- **FillMode**

A drop down list with two options for filling the trail. Fill just draws the trail normally and Wireframe draws lines to outline the polygons of the trail.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **SectionInterval**

The time spans between trail segments, in seconds. Every time this interval is hit a new trail segment is used.

- **SectionLifespan**

This represents the lifetime of each trail segment in seconds.

- **Texture**

The PolyTrail can use a .DTX as its file for the trail to display. To select a .DTX file, click the button and select the file through the Resource Browser.

- **TrailLen**

This is the Length of the trail given by number of segments. For a straight line trail this should be 1 but for a

very curvy trail this should be much higher.

- **TrailWidth**

This is the width of the trail. Make this small for effects like sparks.

- **UAdd**

An offset for the texture coordinates.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position:

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

- **WidthStyle**

This property is a drop down list of predefined ways to draw the trail:

- **Constant**

The trail is always at the constant width set by TrailWidth.

- **SmallToBig**

The trail starts small and ends big.

- **SmallToSmall**

The trail starts small, is big in the middle, and is small at the end again.

- **BigToSmall**

The trail starts big and ends small.

[Top](#)

Sprite

The Sprite FX displays a sprite specified through a .SPR file. The sprite can rotate, be flat against a wall, or always face the camera.

There are many uses for a sprite, one of the most common is to put a rotation on it and use it as a smoke or gas cloud. You can also use it as a smoke puff coming from the barrel of a gun.

Key Options

- **Color Keys**

This FX allows you to edit its color keys. The Sprite's color and alpha will change over time by interpolating between the Color Keys.

- **Scale Keys**

This FX allows you to edit its Scale Keys. The Sprite's scale will change over time by interpolating between the scale keys.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The Sprite will follow the path of the Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **BlendMode**

Depending on the sprite used you may wish to set it to use Additive or Multiply blending.

- **DisableLight**

This controls whether or not dynamic lights affect the color of the sprite.

- **DisableZ**

When this is set to Yes the sprite will look as though it is being drawn over everything else.

- **Facing**

A dropdown list of presets that tell how the sprite will be oriented.

- **CameraFacing**

The sprite will always be oriented so its forward is facing the camera.

- **AlongNormal**

The sprite will face in the direction specified by the Normal Property.

- **ParentAlign**

If the FX has a parent then the sprite will have the same rotation as the parent object.

- **Normal**

This is the normal the sprite will be facing when set to face along the normal. By default this is set to be the forward vector of the sprite.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **Sprite**

This is the sprite (.SPR) file used for the Sprite FX. To select a .SPR file click the button and browse through the Resource Browser and select the desired file.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position.

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

SpriteSystem

This FX is almost identical to the ParticleSystem FX except that the Sprites can rotate about their own axis. The Sprite system emits individual sprites based on the properties that you fill out. This FX allows you to edit its Color Keys, Scale Keys and Motion Keys. The Color Keys and Scale Keys for this FX are per sprite, not per system.

This means the sprite will interpolate between its scale and color keys based on its own lifetime.

The SpriteSystem can be used almost anywhere that a Particle system can be used for and more. The major difference of the Sprite system is that the individual sprites can rotate which can create a sense of movement in smoke and gas clouds. Typically, ParticleSystems look and better and give better performance, but for things like

gas clouds and smoke puffs, adding a rotation makes all the difference in the world.

Key Options

- **Color Keys**

This FX allows you to edit its Color Keys. The Color Keys for a Sprite System are per sprite. So every sprite will use the color keys based on their own lifespan.

- **Scale Keys**

This FX allows you to edit its Scale Keys. The Scale Keys for a Sprite System are per Sprite. So every sprite will use the scale keys based on their own lifespan.

- **Motion Keys**

This FX allows you to edit its Motion Keys. The Sprite System will follow the path of the Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **AType**

A dropdown list for determining the additive type to use for the sprites. Norm uses no additive blending and Add uses additive to render the sprites.

- **EmissionInterval**

This controls the length of time, in seconds, between emissions of the system. Typically this is set low so the system is constantly emitting sprites; however if you want sprites to be emitted in bursts, raise the value.

- **EmissionPlane**

This is the normal vector of the plane that the sprites are emitted from. By default this is a plane parallel to the systems Up vector. If you are using a SpriteSystem as an impact FX this should be set to 0.0 0.0 1.0.

- **EmissionType**

A dropdown list box of predefined emission types for the system:

- **Sphere**

The sprites will emit in all directions around the center of the system a distance no less than MinRadius from the center and no more than MaxRadius from the center.

- **Point**

The sprites will emit from the same random point on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius from the center. If you wish to have the sprites emit from the direct center of the system make Min/MaxRadius 0.0

- **Circle**

The sprites will emit in a circle on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius from the center. Playing with Min/MaxRadius can yield an emission pattern of a circle, doughnut, or ring.

- **GravityAcceleration**

This controls the speed and direction the sprites will float or sink. A negative value will make the sprites sink to the ground and a positive value will float the sprites up towards the sky. Gravity is always in relation to the world, no matter what the systems rotation is.

- **MaxParticleVelocity**

This controls the maximum speed and direction that each sprite will have when they are emitted.

- **MaxSpriteLifeSpan**

Each sprite will last no longer than this amount of time, in seconds. The life span of each sprite is a random number between the Min/MaxSpriteLifeSpan. If you want every Sprite to last the same amount of time, make these exactly the same.

- **MaxRadius**

This is the maximum distance from the center of the system that the sprites will be emitted.

- **MaxSpriteRotation**

The maximum rotation a sprite will rotate around each axis. The sprites will get its rotation by randomizing between Min/MaxSpriteRotation.

- **MinRadius**

This is the minimum distance from the center of the system that the sprites will be emitted.

- **MinSpriteLifeSpan**

Each sprite will last for at least this amount of time, in seconds.

- **MinSpriteRotation**

The minimum rotations a sprite will rotate around each axis.

- **MinSpriteVelocity**

This controls the minimum speed and direction that each sprite will have when they are emitted. The velocity is relative to the system so if you never want sprites to be emitted in the downward direction of the system, make the Y component positive. For impact FX, the Z component should be positive so sprites never get emitted into the surface they hit.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **ParticlesPerEmission**

Every time the system emits sprites, based on the “EmissionInterval”, it will emit this many sprites.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **Sprite**

The SpriteSystem can use a .SPR file as its sprite. Each sprite will loop through the textures in the sprite. To select a ,SPR file, click the button and browse through the Resource Browser and select the desired file.

- **StretchU**

An offset to use to stretch the textures of the sprite.

- **StretchV**

An offset to use to stretch the textures of the sprite.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position.

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

[Top](#)

WonkyFX

This FX plays with the FOV of the camera to create a wavy, watery effect. The cameras X, and Y FOV increases and decreases.

The WonkyFX probably wouldn't be used very much and might only be useful for underwater type effects, or hallucination type effects.

Key Options

- **Color Keys**

This FX does not allow you to edit its Color Keys.

- **Scale Keys**

This FX does not allow you to edit its Scale Keys.

- **Motion Keys**

This FX does not allow you to edit its Motion Keys.

Properties

- **AttachName**

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

- **InnerRadius**

If the player is within this distance from the center of the FX they will feel the wobble effect at full force.

- **Offset**

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

- **OuterRadius**

If the player is between the OuterRadius and the InnerRadius they will feel an interpolated wobble. If outside this distance the player will not feel the wobble.

- **RotateAdd**

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

- **UpdateInterval**

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

- **UpdatePos**

This property is a drop down list of predefined ways to update the FX position.

- **Fixed**

This is the most common option and will keep the FX at a fixed position.

- **Follow**

If the FX is motion linked to another FX UpdatePos must be set to follow.

- **PlayerView**

If you are using this FX as a MuzzleFlash, select PlayerView.

- **NodeAttach**

If this is selected the FX will attach to the specified node.

- **SocketAttach**

If this is selected the FX will attach to the specified socket.

- **PV_NodeAttach**

This will attach the FX to a node in a PlayerView model.

- **PV_SocketAttach**

This will attach the FX to a socket on a PlayerView model.

- **xMultiplier**

This controls the amount of fluctuation in the xFOV. Increasing this value increases how far the xFOV stretches.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Glossary

This section contains a list of terms commonly used in developing LithTech games, and video games in general. These terms may be specific to a product or tool, or they may be common terms used in video game development, graphic design, and level design. Use this section to define terms you are not familiar with. If this section does not contain the term you are looking for, then try one of the following definition sites:

- [TechWeb Encyclopedia](#)
- [Acronym Finder](#)

This section is divided into the following terminology sub-sections:

- [Game Development Terms](#)
 - [Acronyms](#)
 - [File Extensions](#)
-

Game Development Terms

Brush	In DEdit, the brush is the basic unit of world geometry. A brush is made up of planes that define its faces, lines that define its edges, and vertices that define its corners. You can manipulate either an entire brush at once or any of the vertices, lines, or planes in the brush.
Basis Vector	Only used for per-pixel lighting (bump mapping). Specifies the tangent space (or texture space) at each vertex allowing you to put lights in the texture space.
Chroma Key	Term for the alpha mask "layer" applied to textures (.DTX files). Chroma keys are specific colors that are not rendered and appear transparent. Textures with alpha layers get used to create a variety of effects, such as a fencing or windows with bars.
Decal	Decals are textures you can apply over the top of world geometry using the Decal object. Decals operate like spraypaint, covering the geometry of the specified brush(s) in your level.
Flat lighting	In Flat lighting, Processor ray-casts to the vertices of the face currently being lit and then samples the light value of each vertex. It then averages those values and creates a single uniform color across the whole brush. This is fast to process and fast to render, but often ugly.
Game Objects	A resource type. Game objects are similar to .DEP simulation objects, but are constructed for a specific game.
Gouraud shading	Gouraud shading does the same ray-cast to the vertices of the brush as is done for Flat shading, but instead of averaging the colors, it blends their values together gradually, so that you get varying shades that spready out from the vertices to the center of the polygon, creating a smooth light effect. It can be very useful for smoothing sorfaces that would normally look faceted when lightmapped. It is the most common lighting mode for brushes in the Jupiter engine and although it's only average speed when processed, it is as fast as flat shading when rendered, which is to say very fast.
Interpolation	Refers to the computer estimating or filling in data between two points. For instance, in an animation, interpolation occurs between two images to calculate and display the appropriate states of the moving object between the start state and the end state.
Lambert lighting	In the Lambertian lighting model (which is closer to the real world than traditional game lighting models) the amount of light reflected by a surface depends on the ray of light's angle of incidence with the surface it's hitting. The closer to dead on (90 degrees) the angle is, the more light will be reflected from the surface. The closer the ray is to being parallel with the surface, the less light the surface will reflect. Thus, light fades more rapidly from a point source near a large flat surface, and it falls off more rapidly on the rounded sides of a column as well. Usually this is applied to Gouraud-shaded polygons. However, you can also apply it to your lightmaps as well using the Lambertian Lightmaps checkbox. This is a new feature introduced in Jupiter.
Level	See World .
Lightmaps	Lightmaps are a texture layer of relatively low resolution compared to the parent texture that's alpha-blended onto the world polygons to create lighting. It can be highly detailed because it doesn't rely on vertices for color storage. However, since it uses bitmapped

textures, lightmapping requires an additional rendering pass per polygon and also uses much more memory than the previous two. It is expensive to render and also to process, but its detailed shadows mean it's still one of the preferred lighting modes.

Mode	DEdit has several different editing modes: Object, Brush, and Geometry. Each mode allows you to change certain elements of your world (that is, just brushes, objects, or vertices) without accidentally affecting others. They're designed to simplify interacting with your world while editing, so it's important to choose the right mode for your task.
Models	Models in Jupiter are in a custom format, the .LTA file. This contains the mesh/geometry for the model and other information used by the game engine. You create these files in an external editor such as Maya or 3D Studio Max, then save them in the .LTA file format using a plug-in. Use ModelEdit to modify files in the .LTA format once they've been created.
Objects	In DEdit, anything a player interacts with that moves, lights up, shoots, growls, or goes into the player's inventory is an object. DEdit allows you to place any kind of object that your game code or the engine code can create.
Prefab	In DEdit, aprefab is like a primitive, but more complex. If you build a street lamp complete with textures and a light source that gives off just the right shade of light, you can select the lamp and its associated source (or any group of objects and brushes inside DEdit) and save it as a prefab. Thereafter, you can copy and paste that streetlamp into your world without having to rebuild the whole object.
Primitive	In DEdit, a prefab is a simple shape that you can add to the world as the foundation of a more complex shape. Typical primitives are things such as cubes, pyramids, and cylinders.
Processing	In DEdit, when you process your world, a second version of the world is created with some information removed (such as parts that only you need to know about), and other information added (such as parts only Jupiter needs). Processing is a necessary step in getting your world up and running in the game, and is often the first place where you will discover problems with your level.
Project	In DEdit, a project represents the sum of all the resources in the game: all the code, all the textures, all the sounds, all the worlds, and so on. Unlike other world geometry editors, DEdit first opens the project, rather than a world file.
Render Styles	LithTech Render Styles provide a consistent mechanism to customize how models are rendered. The customization encompasses parts of both lighting and rasterization from the basic render states and texture passes to dynamic UV generation and lighting material properties. Programmers can use Render Styles Editor to create unique render styles that enable visually stunning models. Artists can use the Render Styles Viewer to preview any render styles that you apply to your model. Jupiter includes numerous render styles located in the \Development\Tools\RenderStyles\ folder.
Shadow meshing	Shadow meshing is an abandoned method for creating extra polygons for use in Gouraud shading on the world on the fly. If you have a specific use for a feature that will automatically split polygons according to their light and shadows, ask us for more details. However, this code is officially not being developed, so we would have to work with you to make sure it fits your needs and that you can maintain any changes you need to make to it.
Simulation Objects	Objects that programmers construct using code. Some of this code exists in the engine and doesn't appear in the project directories. However, many objects are created in code written for the specific game. These objects are called game objects (to distinguish them from the engine objects inside the engine code) and are stored in a file called OBJECT.LTO in the same directory as the project's .DEP file.
Sprites	Sprites (.SPR files) consist of a series of .DTX files linked together as an animation

with a set frame rate. They're commonly used for animations and special effects such as smoke, bullet holes, and liquid droplets.

Super-sampling	Super-sampling forces Processor to take multiple passes at each lightmap pixel and incorporate a bit of the data from neighboring pixels. Using super-sampling can be expensive (especially if you have a sampling rate of 4 or more) but it results in much smoother-looking lightmaps. Supersampling can almost eliminate jagged stair-stepping in lightmaps. A value of 1-4 is common.
Texture	In DEdit, textures are used like wallpaper, paint, or plaster to cover the raw plywood of your walls, floors, ceilings, doors and so forth. Without a texture, your brush will appear flat-shaded in the game, almost always with unpleasant results. Just as you wouldn't want a house with raw plywood walls, you always want to apply textures to the brushes in your level. You can create .tga textures in a variety of different raster based pixel editors and import them into DEdit where they are converted to .dtx files.
Texture Flags	Texture flags specify a texture definition contained in the surface.txt file. Use texture flags to determine the attributes for a specific texture. For more informaton, see Assigning Attributes to Textures .
Texture Clamping	To disable bilinear filtering for a specific texture or face.
Unit	The basis of measurement in DEdit and Jupiter. Units don't equate directly to real-world values. Instead, the game designers can select a scale of game units to real-world measurements. As an example, in No One Lives Forever 2 the following values are used: railings are 48 units tall, a chest-high box would be 64 units, and a typical doorway would be 128 units high.
Weight Sets	Used in weighted animation blending, weight sets allow you to create custom blends between two concurrent animations.
World	When your players walk around in your game, it is a world that they're standing in. Worlds divide your game up into sections where different parts of the game take place. In other games, these are sometimes referred to as maps, levels, scenes, or episodes. World-editing is the primary focus of DEdit and each world is stored in a separate file on the hard drive.
World Geometry	A Jupiter world is composed of several different classes of geometry including: basic brushes, terrain brushes, WorldModels, and other types such as triggers, special effects, and sky.
World Models	World Models are applied to objects that need some of the properties of models (translucency, mobility, destructibility) but need to be closely physically fitted to a place in the world, or need per-polygon collisions. They are used for doors, windows, elevators and moving machinery.

[Top](#)

Acronyms

BSP	Binary Space Partition—A BSP Tree is structure used to determine the visibility of objects in the game and allow increased game speed by rendering only what is visible to the player.
FFD	Free Form Deformation—A form of soft object animation where the vertices within a mesh are altered to create an animation. FFDs do not export well for use with LithTech.
FOV	Field of View—A property in some lighting objects (such as DirLight) that creates an arc containing the light. Chaning the FOV property widens or narrows the width of the light displayed by the

object.

FPS	First Person Shooter—A game in which the player takes the first person perspective. This acronym is commonly applied to most games involving the player shooting through a viewing window that represents what their character sees.
HUD	Heads Up Display—A display or interface that supplies information and provides controls. Typically HUDs are used to show character status, equipment, and targeting reticles.
LOD	Level of Detail—Refers to the polygon count and how realistic objects look when they are rendered. The higher the polygon count, the higher the level of detail. A delicate balance exists between LOD and the speed allowed by system resources to process the game.
MEL	Maya Embedded Language—The scripting language used by the Maya application. LithTech has developed plugins for Maya that you can access through the MEL interface.
MMP	Massively Multi-Player—A term used to indicate games played by multiple players over an internet or local network connection.
PVS	Potentially Visible Set—
UVW	Not an acronym, but representative of the X/Y/Z coordinates for a texture. UVW mapping is done to determine how a texture gets projected on a polygon.

[Top](#)

File Extensions

.BUT	Designates an ASCII text files containing data used by different classes and objects.
.CFG	Designates a configuration file.
.CFV	Designates a list of stored color animations favorites used by the FxEd tool.
.DAT	Designates a compiled World file created and built in DEdit from an LTA file. DAT files cannot be edited, only run.
.DEP	Designates a DEdit project containing references to all of the objects and components of your world.
.DTX	Designates a texture file created by importing a .PCX or .TGA file to DEdit and saving it as a texture. DTX files also contain flags used by the engine and the game. Textures can be any size up to 32 bits.
.FCF	Designates an FxEd file. These files are automatically generated when you save the current effects project.
.FXD	Designates the FxEd .DLL file containing all the game-side code for Jupiter's special effects.
.FXF	Designates a listing of all the stored special effects created by designers using FxEd.
.KFV	Designates the FxEd list of stored animation key favorites.
.LTA	Prefabs, Worlds, and models all use the .LTA file extension. World .LTA files contain the uncompiled editable source code for a World. World LTA files are edited using DEdit and compiled into .dat files when you build them. Prefab .LTA files contain a collection of brushes and code objects. Prefab.LTA files are created and edited with DEdit. Model .LTA files are created in Maya or Max and exported using the appropriate plugin.

Do not confuse World and Prefab .LTA files with model .LTA files, if you attempt to open a model .LTA file in DEdit, you may encounter an error.

- .LTB** Designates a LithTech binary file. These files contain packed models that have been optimized for display in the game. ModelEdit generates .LTB files. You cannot edit or modify these files, you can only replace them by recompiling.
- .LTC** Designates an .LTA file that has been compressed to save disk space. This format does not save space in the final game or increase performance. They are intended to enable designers to load worlds faster and reduce the amount of space taken up on their machines. .LTC files are typically used for world files.
- .LTO** Designates a file (commonly called the OBJECT.LTO file) containing game objects (similar to simulation objects but created for a specific game).
- .LYT** Designates a log file used by DEdit to store information about the state of a project, when it was last opened, which levels were opened, any various other properties. DEdit uses this file when it opens a world to set the view and settings of the world back to their state when the world was last saved.
- .MEL** Designates a file containing a Maya Embedded Language script. Files with this extension are associated exclusively with the Maya application. For more information about MEL files, consult the Maya documentation.
- .MFV** Designates a list of stored motion type favorites in the FxEd tool.
- .PCX** A common raster-based file format for graphic images.
- .PS** Designates a postscript file.
- .REZ** Designates a LithTech resource file. The lithrez.exe tool creates these files to contain all the files from a project. .REZ files record the directory structure and hierarchy of your project, and they are commonly used to contain files for the retail version of a game. This prevents users from tampering with the retail versions, and also makes installation and file tracking easier. It is not possible to easily edit files inside a .REZ file, so they are typically made as a part of the build and testing process near the end of a project.
- .RPD** Designates a re-processing data file. .RPD files get used by the processor to store information about a level it has processed. When you re-process a level, the processor loads the .RPD file to skip the re-processing of data that has already been processed (although it may not always be able to skip the processing depending on the changes that you have made to your level). .RPD files are created each time you process a level.
- .SFV** Designates a list of stored scale and size animation favorites for use by the FxEd tool.
- .SPR** Designates a sprite file containing a series of .DTX files linked together with a set frame rate.
- .TFG** Designates a Texture Group Effect. .TFG files get generated by DEdit, and store the name of the texture script and texture parameters. .TFG files allow you to apply the same texture effect settings to multiple brushes in your levels.
- .TFS** Designates a Texture Effect Script file. .TGS files are compiled binary scripts containing the run-time information for a single texture or surface effect. Examples include panning, rotation, and scaling of a texture on a brush face. .TFS files are generated by the TSCompiler.exe tool.
- .TGA** Texture files. The .TGA graphic file format is used by many raster-based pixel editors such as Photoshop and PaintshopPro. You can import .TGA files into DEdit and use them as textures.

[Top](#)

[Send feedback regarding this page.](#)
2006, All Rights Reserved.